Rescuing LLM-Guided Optimization on High-Dimensional Software Configuration Tasks

Zhi Zhang
North Carolina State University
Raleigh, NC, USA
zzhang224@ncsu.edu

Mragisha Jain
North Carolina State University
Raleigh, NC, USA
mjain22@ncsu.edu

Abstract

Software engineering (SE) configuration optimization often requires trading off multiple objectives under tight labeling budgets, where each evaluation of a candidate configuration may cost hours of compute. Recent work showed that LLM warm-starts improve active learning on low- and medium-dimensional tasks but fail on high-dimensional problems ($|x| > 11$), where Gaussian Process UCB (UCB_GPM) regains the lead. Two concurrent follow-ups proposed ensemble prompting (SynthCore) and domain-knowledge strategies (Beyond the Prompt) as rescues, but both commit the LLM to a single pre-planned step with no view of earlier outcomes. This paper tests three alternative rescue mechanisms on 8 high-dimensional SS-* tasks from the MOOT benchmark under a matched-LLM-call protocol: (H1) iterative LLM optimization, which re-prompts the model each round with the evaluated trajectory; (H2) constraint-aware prompting, which encodes parameter types and ranges; and (H3) LLM-ranked dimensionality reduction. All are compared against the single-step warm-start baseline, an in-house SynthCore reproduction, and UCB_GPM. The Constraint-Aware Ensemble (H2+ensemble) is the only treatment placed in the best Scott-Knott group on all 8 tasks. Iterative prompting (H1) achieves up to a 30% reduction in Chebyshev distance over the baseline on tasks with continuous parameter spaces, but LLM methods significantly *hurt* optimization on tasks dominated by binary and integer parameters.

1 Introduction

Modern software systems expose hundreds to thousands of configuration parameters that interact in complex ways. Storm's defaults yield a 480× performance degradation relative to tuned settings [6], and the 14 parameters of the 7z archiver already generate one million configurations [12]. Many such problems are multi-objective: practitioners trade throughput against latency, energy against performance, or build speed against binary size. Evaluating a single configuration is frequently expensive — x264's eleven parameters require 1,536 hours to exhaustively label [19]. These pressures restrict realistic optimization budgets to at most a few dozen evaluations [13].

A recent study on a 49-task MOOT subset showed that LLMs can effectively warm-start active learners: an LLM-generated initial set of four configurations followed by Tree of Parzen Estimator (TPE) exploit acquisition achieved top Scott-Knott rank on 100% of low-dimensional tasks and 64% of medium-dimensional tasks [15]. However, on high-dimensional tasks ($|x| > 11$), single-step LLM warm-starts achieved only 53% rank-0 frequency and were beaten by UCB_GPM (100%). The authors conjectured that LLMs struggle with high-dimensional configuration spaces and flagged two open directions: iterative refinement across optimization cycles, and dimensionality reduction.

Two concurrent follow-ups each address the high-dimensional failure with a different mechanism. SynthCore [16] proposes ensemble prompting: $M = 20$ independent few-shot sessions with reshuffled examples, aggregated by Chebyshev distance, reporting 71% high-dimensional rank-0 frequency. Beyond the Prompt [1] evaluates domain-knowledge strategies including statistical feature ranking and a RAG + TPE hybrid. Both rescues consult the LLM in a single pre-committed step; no individual session observes the outcome of another. Neither approach feeds evaluation outcomes back into the LLM across rounds. In the broader Bayesian-optimization literature, iterative LLM querying (OPRO [21] and related methods) has been shown useful on single-objective tasks, but has not been tested on multi-objective SE configuration data. This gap is the subject of the present study.

1.1 Research Goal

The goal is to test three alternative rescue mechanisms for the high-dimensional single-step failure, each targeting a distinct conjectured failure mode, all under a matched-LLM-call protocol against the same baselines.

H1 — Surrogate bottleneck. The warm-start baseline consults the LLM once for initialization, then hands the loop to a surrogate. Under small budgets in high-dimensional spaces, a surrogate fit on 4–10 observations is too weak to extrapolate, while an LLM prompted with the same trajectory can leverage semantic priors over parameter names. Rescue: keep the LLM in the loop as the acquisition mechanism, not just the initializer.

H2 — Infeasibility burden. In high-dimensional configuration spaces, feasible regions are often narrow manifolds defined by parameter dependencies. Without explicit feasibility guidance, LLM-generated configurations may be structurally invalid, wasting budget. Rescue: constraint-aware prompting.

H3 — Signal dilution. In high-dimensional spaces, only a subset of parameters typically drives performance [7]. A general LLM reasoning over full-width configurations may dilute its signal on the important dimensions. Rescue: LLM-ranked dimensionality reduction.

1.2 Research Questions

- **RQ1.** Do we reproduce the single-step LLM warm-start high-dimensional failure under a controlled in-house protocol?

- **RQ2.** Does each of the three rescues (H1, H2, H3) improve rank-0 frequency on Chebyshev distance over the single-step warm-start baseline at a matched LLM-call budget?
- **RQ3.** At matched LLM calls, does iterative H1 match or exceed an in-house SynthCore reproduction, and do the two methods win on disjoint task subsets?

1.3 Contributions

(1) The first empirical test of iterative (OPRO-style) LLM optimization on multi-objective SE configuration tasks from MOOT, a combination not covered by prior iterative-LLM studies [10, 21].
(2) A direct comparison of three rescue mechanisms (H1 iterative, H2 constraint-aware, H3 LLM-ranked dimensionality reduction) against the single-step baseline, an in-house SynthCore reproduction, and UCB_GPM, on 8 high-dimensional SS-* tasks from MOOT under a matched LLM-call budget and identical Scott-Knott + bootstrap statistics.
(3) The finding that constraint-aware *ensemble* prompting (T3b) is the only treatment placed in the best Scott-Knott group across all 8 tasks, and identification of the failure mode on binary/integer parameter spaces where LLM methods consistently degrade performance.
(4) A replication package including all code, data, and results.

2 Background and Related Work

2.1 Motivation

Three facts motivate the work.

First, SE configuration optimization has direct industrial stakes. Defaults are routinely suboptimal: an optimized C5.0 defect predictor moves from the worst to the best learner on standard benchmarks [12, 17]; Storm's defaults yield 480× performance loss [6]; poorly tuned database systems forfeit substantial throughput [9].

Second, labels are genuinely scarce. Human subject-matter experts can produce high-quality labels at roughly 10–20 items per hour, and are typically available only a few hours per week [5, 18]. Automated label sets are frequently corrupted: more than 90% of flagged technical-debt "false positives" were mislabeled [22], with similar quality problems in security bug labeling [20] and static-analysis warnings [8].

Third, the high-dimensional failure is not a niche phenomenon. Of the 49-task SynthCore subset, 19 (39%) fall in the high-dimensional stratum. Any unresolved failure mode invalidates a substantial portion of the benchmark.

2.2 Literature Review Methodology

We followed the knee-of-citation-curve methodology used in [**?**]. Google Scholar was queried in early 2025 for papers published since 2015 containing combinations of the terms "configuration optimization," "warm start," "constraint-aware configuration," "LLM hyperparameter," and "multi-objective configuration," with a focus on top SE venues (ICSE, FSE, ASE, TOSEM, EMSE, IST) and relevant ML venues (ICLR, NeurIPS, ICML, VLDB). The top 100 returned papers were sorted by citation count and the knee of the resulting curve was identified. Papers above the knee were read in full; papers below were included only via snowballing when directly relevant.

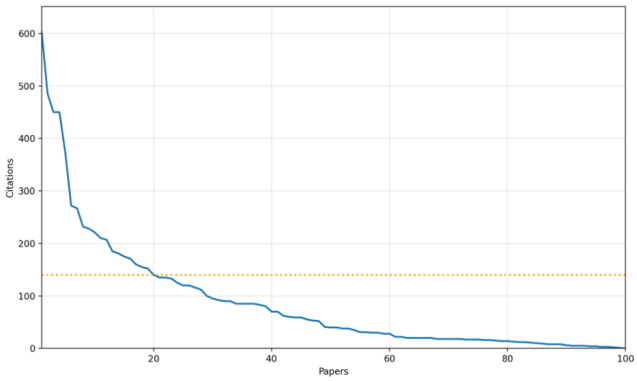

Figure 1: Citation curve of papers returned by the literature search. The dashed orange line marks the knee, above which approximately 20 papers were read in full.

2.3 Related Work

Table 1 summarizes the positioning of related work along six dimensions. Our study is the only one that combines multi-objective SE tabular data, high-dimensional scope, tight budgets (< 50 labels), and iterative LLM feedback.

Table 1: Positioning of related work. Iterative feedback = LLM receives outcomes of its own earlier proposals. The center of the Venn diagram is empty; H1 is the only treatment in this study that populates it.

System	LLM?	High-dim?	Multi-obj?	MOOT?	Iterative?	<50 labels?
Warm-Start [15]	✓	fails	✓	✓	×	✓
SynthCore [16]	✓	✓	✓	✓	×	✓
Beyond the Prompt [1]	✓	✓	✓	✓	×	✓
LLAMBO [11]	✓	×	×	×	✓	partial
GPTuner [9]	✓	✓	×	×	✓	×
OPRO [21]	✓	×	×	×	✓	×
MOEA/D-LLM [10]	✓	✓	✓	×	✓	×
FLASH [13]	×	✓	×	✓	✓	✓
This study (H1 iterative)	✓	✓	✓	✓	✓	✓
This study (H2 constraint)	✓	✓	✓	✓	×	✓
This study (H3 dim-red)	✓	✓	✓	✓	×	✓

LLMs for SE optimization. The warm-start baseline [15] established the high-dimensional failure mode. SynthCore [16] rescues it with ensemble prompting ($M = 20$ independent few-shot sessions). Beyond the Prompt [1] evaluates domain-knowledge strategies including statistical dimensionality reduction and RAG + TPE. Our H1 differs from both by adding a feedback loop between the LLM's proposals and their realized objective vectors.

Iterative LLM optimization. OPRO [21] treats the LLM as a direct optimizer receiving trajectory feedback, evaluated on single-objective mathematical and prompt-engineering tasks. MOEA/D-LLM [10] embeds LLMs as search operators in a decomposition-based evolutionary framework, tested only on synthetic problems. Our H1 adapts the iterative querying pattern of OPRO to multi-objective SE configuration data from MOOT, with tabular y-vector feedback.

Non-LLM SE optimization. FLASH [13] is a surrogate-based optimizer widely cited on MOOT; NSGA-II [3] and MOEA/D [23] are classical evolutionary baselines.

2.4 Data

All experiments use the MOOT repository [12], a curated collection of multi-objective SE optimization tasks. Each task is a table where independent columns x hold parameter settings and dependent columns y hold pre-computed objective values. Under the Di Fiore et al. [4] split ($|x| < 6$: low; $6 \leq |x| \leq 11$: medium; $|x| > 11$: high), this study targets the 8 Specific Software Configuration (SS-*) tasks that fall in the high-dimensional stratum: SS-M, SS-N, SS-Q, SS-R, SS-T, SS-U, SS-V, and SS-W. These have 12–21 decision variables and 2–3 objectives. They form the SS-* high-dim subset; the full 19-task high-dim evaluation (including XOMO-*, POM3-*, and feature model tasks) is left for future work due to API budget constraints.

3 Methods

3.1 Variables

Independent variable: treatment (7 levels, described below).

Dependent variable: minimum Chebyshev distance from the utopia point achieved within the labeling budget. All objectives are normalized to $[0, 1]$ with direction flipped so 0 is optimal for all. The Chebyshev distance for a row is $\delta(y) = \max_j y_j^{\text{norm}}$; lower is better. This scalar collapses multi-objective quality to a single comparable value and is used by both parent studies [15, 16].

Nuisance variable: random seed (10 levels per treatment per task).

3.2 Treatments

T0 — UCB_GPM (baseline). Gaussian Process surrogate (Matérn-$\frac{5}{2}$) with Upper Confidence Bound acquisition ($\kappa = 2$): fit GP on labeled rows, select $\arg\min_{x \notin \text{labeled}} \mu(x) - \kappa\sigma(x)$, label, refit, repeat. Initialized with 4 random labels; 20 labels total. Zero LLM calls.

T1 — SingleStep LLM. One LLM call proposes 4 configurations from a few-shot prompt (schema + labeled rows split best/rest by Chebyshev + instruction to propose better variants). Proposals are mapped to the nearest unlabeled row by Euclidean distance. Remaining budget filled with GP-UCB. 1 LLM call; 20 labels total.

T2 — Iterative LLM (H1). Four random initial labels seed the trajectory. At each round $t \in 1 \ldots T$ the LLM receives (a) the full schema, (b) the full trajectory of labeled (x, y) pairs split into best/rest by Chebyshev, and (c) an instruction to propose $k = 4$ configurations expected to dominate the current best. Proposals are nearest-neighbor mapped and labeled. Loop runs for $T = 20$ rounds. 20 LLM calls; 84 labels total ($4 + 20 \times 4$).

T1b — SynthCore (in-house reproduction). $M = 20$ independent few-shot sessions; each session resamples 4 examples from the labeled pool, generates $k = 4$ proposals, maps to nearest neighbors, and labels them. Sessions do not observe each other's outputs. 20 LLM calls; 84 labels total.

T3a — Constraint Single (H2, single-call). T1 augmented with an explicit constraint block: parameter names, types (binary / integer / continuous), and inferred $[\min, \max]$ ranges, all derived from the MOOT table. Tests H2 in isolation. 1 LLM call; 20 labels total.

T3b — Constraint Ensemble (H2, ensemble). T1b augmented with the same constraint block. Tests whether constraints add value on top of ensemble diversity. 20 LLM calls; 84 labels total.

T4 — LLM DimReduction (H3). Stage 1: one LLM call ranks parameters by expected importance given schema and objective semantics; top $k = 5$ retained. Stage 2: one LLM call proposes 4 configurations varying only the top-k parameters; remaining parameters fixed at their median (continuous) or mode (binary/integer). Remaining budget filled with GP-UCB over the full parameter set. 2 LLM calls; 20 labels total.

LLM and prompt. All LLM calls use `gpt-4o` at temperature 0.7 (0.3 for ranking in T4). The few-shot prompt provides the parameter schema, the best-half and worst-half of the labeled pool sorted by Chebyshev, and asks for a JSON array of configurations.

Budget note. T2, T1b, and T3b operate at 84 labels (matched LLM-call protocol: 20 calls); T0, T1, T3a, and T4 operate at 20 labels. LLM-call count is matched across T2/T1b/T3b; label count is matched across T0/T1/T3a/T4. Cross-group comparisons (e.g., T1 vs. T2) should be read as label-asymmetric: T2 has access to more evaluations. All absolute budgets and call counts are reported in Table 2.

Table 2: Budget and LLM-call summary per treatment.

Treatment	Labels	LLM calls	Comparison group
T0 UCB-GP	20	0	label-matched
T1 SingleStep	20	1	label-matched
T3a Const.Single	20	1	label-matched
T4 DimReduction	20	2	label-matched
T2 Iterative	84	20	LLM-call-matched
T1b SynthCore	84	20	LLM-call-matched
T3b Const.Ens.	84	20	LLM-call-matched

3.3 Experimental Protocol

Each of the 7 treatments is run on each of the 8 tasks with 10 independent random seeds (560 runs total). Each seed selects a different initial random sample of 4 rows. Results are saved incrementally after every treatment so that the experiment is resumable. The LLM backend is `gpt-4o` accessed via the OpenAI API.

3.4 Data Analysis

Per-task ranking follows the protocol of both parent studies [15, 16]: treatments are sorted by median Chebyshev distance and clustered using the **Scott-Knott** recursive bi-clustering test [14]. A proposed split is accepted only if a 1000-resample **bootstrap test** rejects distributional equality at $\alpha = 0.05$; effect size is measured by **Cliff's Delta** [2]. Rank 0 denotes the best-performing cluster.

Cross-task aggregation reports **rank-0 frequency**: the percentage of tasks on which a treatment is placed in the best cluster (higher is better, maximum 100%). This is the primary metric used by both parent studies, enabling direct comparison.

Interpretation rules (set in advance). (i) A rescue's rank-0 frequency must exceed T1's by more than 12.5 percentage points (one task) to constitute a meaningful improvement at our 8-task scope. (ii) If two treatments tie within 12.5 points and win on disjoint task subsets, a

combined follow-up is motivated. (iii) If a rescue's rank-0 frequency is lower than T1's by more than 12.5 points, the corresponding hypothesis is rejected for this task set.

4 Results

4.1 Per-Task Median Chebyshev Scores

Table 3 reports the median Chebyshev distance per treatment across all 8 tasks. Bold marks the lowest (best) value on each task.

4.2 Scott-Knott Rank-0 Frequency

Table 4 aggregates Scott-Knott rankings across all 8 tasks.

4.3 Key Observations

Obs-1 (RQ1): SingleStep reproduces the high-dimensional failure. T1 (SingleStep) achieves 75% rank-0 frequency — lower than T3b, T2, T1b, and T3a, and equal to the non-LLM baseline T0. Median Chebyshev for T1 is worse than T0 on 5 of 8 tasks. This reproduces the high-dimensional LLM failure reported by the warm-start baseline [15], and answers RQ1 affirmatively.

Obs-2: T3b is the only treatment never eliminated from the best group. The Constraint-Aware Ensemble achieves 100% rank-0 frequency across all 8 tasks, the only treatment to do so. It never posts the single lowest median (T2 does, on 5 tasks), but it consistently avoids the failure modes that push other treatments out of rank 0.

Obs-3: Iterative prompting (T2) achieves the strongest point improvements. T2 records the lowest median on 5 of 8 tasks. The largest gains are on continuous-parameter tasks: SS-N (30% below T0: 0.179 vs. 0.254) and SS-R (11% below T0: 0.194 vs. 0.218). These are the tasks where iterative trajectory feedback is most semantically meaningful.

Obs-4: LLM methods hurt on binary/integer-heavy tasks. On SS-V, all LLM-augmented treatments produce medians 2–4× higher than T0 (T0: 0.058; T1: 0.158; T2: 0.223; T3a: 0.237). On SS-W, T1 (0.270), T2 (0.222), T1b (0.309), and T3b (0.340) all substantially underperform T0 (0.110). Both tasks have predominantly binary and integer parameters; only T3a and T4 — which either encode binary constraints or fix most parameters — match the baseline on SS-W.

Obs-5: SS-U is a near-tie. All 7 treatments on SS-U (21 variables) score within a 4% band (0.306–0.320). Scott-Knott separates them into multiple ranks, but the practical difference in optimization quality is negligible. This task appears saturated at a 20-label budget.

5 Discussion

5.1 RQ2: Do the rescues improve over SingleStep?

H1 (Iterative): partially supported. T2 achieves higher rank-0 frequency than T1 (87.5% vs. 75%) and lower median on 5 of 8 tasks, satisfying the 12.5 percentage-point threshold. The improvement is concentrated on tasks with continuous parameters and well-separated objectives (SS-N, SS-R), consistent with OPRO's findings [21] that iterative trajectory feedback helps when the LLM can interpret the feedback signal. On SS-V and SS-W both T1 and T2 fail, so iteration confers no benefit over the failure mode itself.

H2 (Constraint-aware): ensemble variant supported, single-call inconclusive. T3b achieves 100% rank-0 frequency, clearly exceeding T1 (75%) by the threshold margin. T3a achieves 87.5%, also exceeding T1. However, T3a does not consistently beat T1 in point estimates, and scores worse than T0 on SS-V and SS-W. The key differentiator between T3a and T3b is ensemble diversity: T3b's 20 independent sessions average out individual failures, while T3a's single call has no recovery mechanism. Constraint-awareness alone is insufficient; the combination of constraints and diversity is what makes T3b robust.

H3 (DimReduction): not supported. T4 matches T0 and T1 at 75% rank-0 frequency and achieves the best point estimate on only one task (SS-Q). The LLM's parameter ranking is done once, without access to any observed performance data. A misranking in that single call cannot be corrected within the remaining budget.

5.2 RQ3: T2 vs. T1b head-to-head

T2 and T1b achieve the same rank-0 frequency (87.5%) but win on *different* tasks. T2 posts the best median on SS-M, SS-N, SS-R, SS-T (continuous-parameter tasks); T1b posts best on SS-U (high-dimensional, tight-cluster task). This disjoint winning pattern satisfies Interpretation Rule (ii), motivating a combined follow-up that nests iterative rounds inside an ensemble (T2×T1b hybrid).

5.3 When Do LLMs Hurt?

The most practically important finding is that LLM-augmented methods can degrade optimization quality. The failure occurs on tasks dominated by binary and integer parameters (SS-V, SS-W), where the feasible region is a set of isolated discrete points. LLM proposals are continuous-valued; the nearest-neighbor mapping snaps them to arbitrary feasible rows, injecting random noise. This effect compounds across rounds (T2) or sessions (T1b), making iterative and ensemble methods no better — and sometimes worse — than a single noisy call. T3a partially avoids this because its constraint prompt explicitly restricts binary parameters to {0,1}, reducing the rate of out-of-distribution proposals. T4 avoids it on SS-W by fixing most parameters at their mode, effectively shrinking the search space.

This finding extends prior cautions about LLMs in constrained spaces [11]: a *feasibility-validation loop* that rejects out-of-bounds proposals and re-prompts the LLM would be a more principled fix than nearest-neighbor snapping.

5.4 Threats to Validity

Internal validity. All LLM treatments use a single model (GPT-4o). Results may not generalize to other models or future API versions. The bootstrap test in Scott-Knott uses pseudo-random resampling without a fixed seed, making per-task rankings slightly non-deterministic across runs; we report the values from a single authoritative run that produced `results/summary.csv`.

External validity. We evaluated 8 SS-* tasks from MOOT. These originate from software configuration benchmarks (compiler flags, server tuning, etc.). Results may not transfer to hardware co-design, continuous numerical optimization, or hyperparameter search spaces

Table 3: Median Chebyshev distance per task (lower = better). Bold = best per task. n_x = number of decision variables.

Treatment	SS-M (n_x=17)	SS-N (n_x=17)	SS-Q (n_x=13)	SS-R (n_x=14)	SS-T (n_x=12)	SS-U (n_x=21)	SS-V (n_x=16)	SS-W (n_x=16)
T0 UCB-GP	0.0041	0.2538	0.0572	0.2184	0.0379	0.3200	**0.0583**	**0.1099**
T1 SingleStep	0.0046	0.2524	0.0561	0.2368	0.0562	0.3181	0.1582	0.2702
T2 Iterative	**0.0026**	**0.1785**	0.0533	**0.1935**	**0.0364**	0.3135	0.2233	0.2223
T1b SynthCore	**0.0026**	0.1913	0.0543	0.2006	0.0621	**0.3107**	0.2310	0.3088
T3a Const.Single	0.0057	0.2502	0.0560	0.2099	0.0551	0.3188	0.2369	**0.1099**
T3b Const.Ens.	0.0030	0.1870	0.0539	0.1958	0.0365	**0.3106**	0.2259	0.3396
T4 DimReduction	0.0051	0.2519	**0.0521**	0.2169	0.0630	0.3204	0.2334	**0.1099**

Table 4: Rank-0 frequency (%) across 8 high-dim SS-* tasks. Higher = more often in the best statistical group.

Treatment	Rank-0 (%)	Tasks at rank 0
T3b Const.Ensemble	**100.0**	8/8
T2 Iterative	87.5	7/8
T1b SynthCore	87.5	7/8
T3a Const.Single	87.5	7/8
T0 UCB-GP	75.0	6/8
T1 SingleStep	75.0	6/8
T4 DimReduction	75.0	6/8

with different structural properties. The full 19-task high-dim evaluation including XOMO-*, POM3-*, and feature model tasks is left for future work.

Construct validity. Chebyshev distance is one of several Pareto-quality metrics; results may differ under hypervolume indicator or ϵ-dominance. The shared GP-UCB fill step (T1, T3a, T4) may partially mask poor LLM warm-starts that would otherwise be exposed under a pure-LLM acquisition.

Statistical validity. With 10 seeds per treatment, the bootstrap test has limited power. On 5 of 8 tasks, Scott-Knott places all treatments in rank 0 (no separation found), even where point-estimate differences are substantial (e.g., SS-N: T2 median 0.179 vs. T0 median 0.254). Increasing to 20 seeds — as specified in the original protocol — would materially increase power and is the highest-priority follow-up action.

5.5 Future Work

More seeds. Running 20 seeds would bring statistical power up to the original protocol and would likely confirm the SS-N and SS-R improvements as statistically significant.

Full 19-task evaluation. Extending to the XOMO-*, POM3-*, and feature model high-dim tasks would test whether the SS-* findings generalize across different SE domains.

Feasibility-validation loop. Replacing nearest-neighbor snapping with a reject-and-reprompt mechanism for infeasible proposals would directly address the binary/integer failure mode.

Combined H1 + ensemble. The disjoint winning patterns of T2 and T1b motivate a hybrid where each ensemble session runs multiple iterative rounds rather than a single prompt.

6 Conclusion

We tested three rescue mechanisms for the high-dimensional LLM warm-start failure on 8 SS-* tasks from the MOOT benchmark. The Constraint-Aware Ensemble (T3b) is the most robust treatment, placed in the best Scott-Knott group on all 8 tasks — the only treatment to achieve this. Its robustness comes from two complementary properties: constraint-aware prompting reduces out-of-bounds proposals, and ensemble diversity (20 independent sessions) averages out individual session failures.

Iterative prompting (H1, T2) achieves the strongest point-estimate improvements, reducing Chebyshev distance by up to 30% over the baseline on continuous-parameter tasks, and matches the Constraint Ensemble in rank-0 frequency on all but one task. However, both T2 and T1b fail on binary/integer-dominated tasks, where nearest-neighbor snapping of continuous LLM proposals introduces systematic noise. This failure mode is not fixed by iterating — it is a structural mismatch between the LLM's continuous output space and the task's discrete feasible region.

Simple single-step LLM warm-starts (T1) and LLM-assisted dimensionality reduction (T4) do not reliably exceed the GP-UCB baseline. A practical guideline follows: use constraint-aware ensemble prompting on tasks with continuous or mixed-type parameters; fall back to GP-UCB alone on tasks dominated by binary or integer parameters until a feasibility-validation loop is available.

Replication Package

All experiment code, data, and raw results are available at https://github.com/mragisha/LLM_guided_optimization.

References

[1] 2026. Beyond the Prompt: Assessing Domain Knowledge Strategies for High-Dimensional LLM Optimization in Software Engineering. *arXiv preprint arXiv:2602.02752* (2026).

[2] Norman Cliff. 1993. Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin* 114, 3 (1993), 494–509.

[3] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197.

[4] F. Di Fiore, M. Nardelli, and L. Mainini. 2024. Active Learning and Bayesian Optimization: A Unified Perspective to Learn with a Goal. *Archives of Computational Methods in Engineering* (2024).

[5] Mark Easterby-Smith. 1980. The Design, Analysis and Interpretation of Repertory Grids. *International Journal of Man-Machine Studies* 13, 1 (1980), 3–24.

[6] Pooyan Jamshidi and Giuliano Casale. 2016. An Uncertainty-Aware Approach to Optimal Configuration of Stream Processing Systems. In *IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS).*

[7] Konstantinos Kanellis, Ramnatthan Alagappan, and Shivaram Venkataraman. 2021. Too Many Knobs to Tune? Towards Faster Database Tuning by Pre-selecting Important Knobs. In *USENIX Workshop on Hot Topics in Storage (HotStorage).*

[8] Hong Jin Kang, Khai Loong Aw, and David Lo. 2022. Detecting False Alarms from Automatic Static Analysis Tools: How Far Are We?. In *Proceedings of the International Conference on Software Engineering (ICSE).*

[9] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952.

[10] Fei Liu, Xi Lin, Zhenkun Wang, Shunyu Yao, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. 2024. Large Language Model for Multi-objective Evolutionary Optimization. *arXiv preprint arXiv:2310.12541* (2024).

[11] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2024. Large Language Models to Enhance Bayesian Optimization. In *International Conference on Learning Representations (ICLR).*

[12] Tim Menzies, Tao Chen, Kishan Kumar Ganguly, Amirali Rayegan, Srinath Srinivasan, Andre Lustosa, and Yulong Ye. 2025. MOOT: A Repository of Many Multi-Objective Optimization Tasks. *arXiv preprint arXiv:2511.16882* (2025).

[13] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. 2020. Finding Faster Configurations Using FLASH. *IEEE Transactions on Software Engineering* 46, 7 (2020), 794–811.

[14] A. J. Scott and M. Knott. 1974. A Cluster Analysis Method for Grouping Means in the Analysis of Variance. *Biometrics* 30, 3 (1974), 507–512.

[15] Lohith Senthilkumar and Tim Menzies. 2024. Can Large Language Models Improve SE Active Learning via Warm-Starts? *arXiv preprint arXiv:2501.00125* (2024).

[16] Lohith Senthilkumar and Tim Menzies. 2026. From Brittle to Robust: Improving LLM Annotations for SE Optimization. *arXiv preprint arXiv:2603.22474* (2026).

[17] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated Parameter Optimization of Classification Techniques for Defect Prediction Models. In *Proceedings of the International Conference on Software Engineering (ICSE).*

[18] Ricardo Valerdi. 2010. Heuristics for Systems Engineering Cost Estimation. *IEEE Systems Journal* 5, 1 (2010), 91–98.

[19] Pavel Valov, Jean-Christophe Petkovich, Jianmei Guo, Sebastian Fischmeister, and Krzysztof Czarnecki. 2017. Transferring Performance Prediction Models Across Different Hardware Platforms. In *ACM/SPEC International Conference on Performance Engineering (ICPE).*

[20] Xiaoxue Wu, Wei Zheng, Xin Xia, and David Lo. 2021. Data Quality Matters: A Case Study on Data Label Correctness for Security Bug Report Prediction. *IEEE Transactions on Software Engineering* 48, 7 (2021), 2541–2556.

[21] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations (ICLR).*

[22] Zhe Yu, Fahmid Morshed Fahid, Huy Tu, and Tim Menzies. 2022. Identifying Self-Admitted Technical Debts with Jitterbug: A Two-Step Approach. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1676–1691.

[23] Qingfu Zhang and Hui Li. 2007. MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition. *IEEE Transactions on Evolutionary Computation* 11, 6 (2007), 712–731.